

Clase 6

RECURSIÓN (SEGUNDA PARTE)

CC1002 Introducción a la Programación

Implementando funciones recursivas

- Son funciones condicionales
- Metodología:
 - Definir el o los casos base
 - Pensar en forma inductiva
 - Llamado recursivo debe llevar a algún caso base
 - Usar la hipótesis de inducción y suponer que el llamado recursivo devuelve lo correcto
 - Pensar qué es lo que le falta a lo devuelto por el llamado recursivo para obtener el resultado final

CC1002 Introducción a la Programación

Receta de diseño para la recursión

- Escribir varios ejemplos de uso de la función (incluyendo parámetros y resultado)
- Decidir cual de los argumentos va a tener una descomposición recursiva
- Entender cuál es el caso de base para este argumento
 - Muy importante que el caso base se alcance en algún momento, sino se produce un error lógico

CC1002 Introducción a la Programación

Receta de diseño para la recursión

- Nombrar cada parte del problema
- Aplicar el problema a uno de los ejemplos
- Ocupar la receta de diseño normal, tomando en cuenta que el patrón de base es una función condicional, una rama siendo el caso de base, la otra siendo el caso recursivo

CC1002 Introducción a la Programación

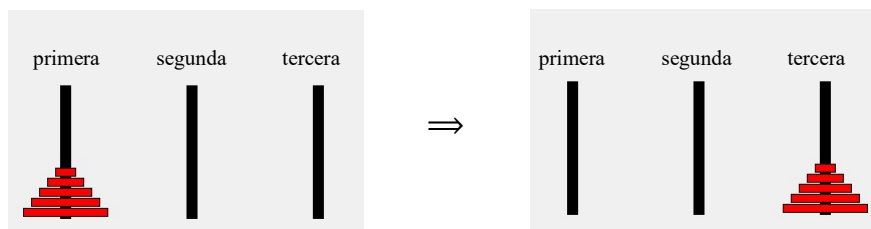
Funciones recursivas

- Ejemplo: Torres de Hanoi
 - Puzzle matemático
 - Se tienen n discos de distinto radio
 - Hay tres torres donde se colocan los discos
 - Reglas:
 - Sólo se puede mover un disco a la vez
 - No puede haber un disco más grande sobre uno más pequeño
 - Movimiento: mover un disco en la cima de una torre hacia la cima de otra torre de discos

CC1002 Introducción a la
Programación

Funciones recursivas

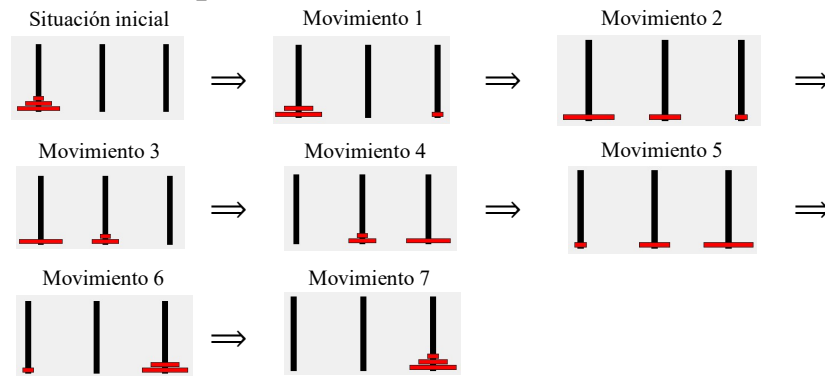
- Ejemplo con $n = 5$ discos



CC1002 Introducción a la
Programación

Funciones recursivas

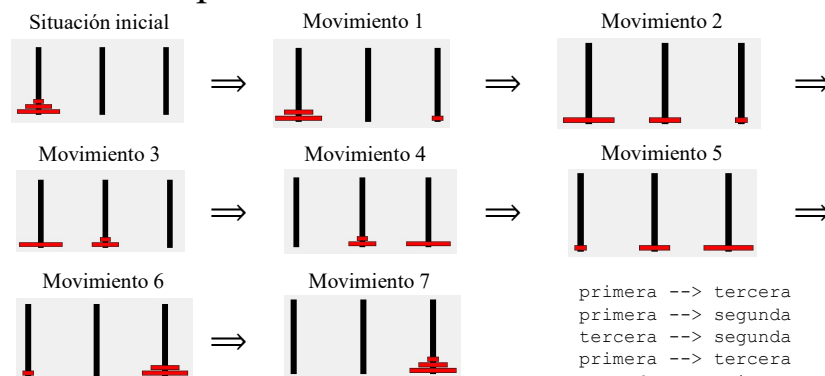
• Solución para $n = 3$ discos



CC1002 Introducción a la Programación

Funciones recursivas

• Solución para $n = 3$ discos



CC1002 Introducción a la Programación

```

primera --> tercera
primera --> segunda
tercera --> segunda
primera --> tercera
segunda --> primera
segunda --> tercera
primera --> tercera

```

Funciones recursivas

- Solución recursiva para n discos:
 - Mover $n - 1$ discos desde la primera hasta la segunda torre, utilizando la tercera torre como torre auxiliar
 - Mover el disco más grande desde la primera a la tercera torre
 - Mover $n - 1$ discos desde la segunda hasta la tercera torre, utilizando la primera torre como torre auxiliar

CC1002 Introducción a la
Programación

Funciones recursivas

```
# hanoiRec: int str str str -> str
# Devuelve un string con la lista de movimientos para
# resolver el problema de las Torres de Hanoi con n discos,
# moviendolos desde la torre de partida a la torre final
# y quedando la torre del medio como torre auxiliar
def hanoiRec(n, partida, medio, final):
    if n == 0: # Caso base
        return ''
    else: # Caso recursivo
        # Se mueven n - 1 discos de torre de partida a torre del medio
        texto1 = hanoiRec(n - 1, partida, final, medio)
        # Se mueve el disco mas grande de torre de partida a torre final
        texto2 = str(partida) + ' --> ' + str(final) + '\n'
        # Se mueven n - 1 discos de torre del medio a torre final
        texto3 = hanoiRec(n - 1, medio, partida, final)
        return texto1 + texto2 + texto3

# hanoi: int -> str
# Invoca el primer llamado recursivo de hanoiRec
# ejemplo: hanoi(4) devuelve el texto con los movimientos para 4 discos
def hanoi(n):
    return hanoiRec(n, 'primera', 'segunda', 'tercera')
```

CC1002 Introducción a la
Programación

Ejemplo: fractales

```
import turtle

turtle.hideturtle()
turtle.speed(0)

# Tortuga parte al centro mirando a la derecha
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.right(90)
turtle.penup()
turtle.forward(100)
turtle.pendown()
turtle.forward(100)
turtle.left(90)
turtle.backward(100)
turtle.done()
```

CC1002 Introducción a la
Programación

Ejemplo: fractales

```
import turtle

turtle.hideturtle()
turtle.speed(0)

# triangulo: int -> None
# dibuja un triangulo equilatero
# de lado largo
def triangulo(largo):
    turtle.forward(largo)
    turtle.left(120)
    turtle.forward(largo)
    turtle.left(120)
    turtle.forward(largo)
    turtle.left(120)

# sierpinski: int -> None
# dibuja el triangulo de Sierpinski
# de longitud largo
def sierpinski(largo):
    if largo > 4:
        triangulo(largo)
        sierpinski(largo // 2)
        turtle.forward(largo // 2)
        sierpinski(largo // 2)
        turtle.backward(largo // 2)
        turtle.left(60)
        turtle.forward(largo // 2)
        turtle.right(60)
        sierpinski(largo // 2)
        turtle.left(60)
        turtle.backward(largo // 2)
        turtle.right(60)

# Triangulo de Sierpinski
sierpinski(256)
turtle.done()
```

CC1002 Introducción a la
Programación

Para la próxima clase

- Leer el Capítulo 7 del apunte

CC1002 Introducción a la
Programación